Verification and Validation Report: Bridging Gaps: AI for Diagram Accessibility

Team 22, Reading4All
Nawaal Fatima
Dhruv Sardana
Fiza Sehar
Moly Mikhail
Casey Francine Bulaclac

March 9, 2026

1 Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

2 Symbols, Abbreviations, and Acronyms

Table 1: Abbreviations and Acronyms Used in the Verification and Validation Plan

Acronym	Definition
AI	Artificial Intelligence
API	Application Programming Interface
AODA	Accessibility for Ontarians with Disabilities Act
AR	Accessibility Requirement
CAD	Canadian Dollar
CI	Continuous Integration
CD	Continuous Development
CR	Cultural/Regulatory Requirement
DP	Development Plan
FR	Functional Requirement
GUI	Graphical User Interface
HA	Hazard Analysis
LFR	Look and Feel Requirement
LR	Legal Requirement
MG	Module Guide
MIS	Module Interface Specification
ML	Machine Learning
MS	Maintenance Support Requirement
NFR	Nonfunctional Requirement
NVDA	NonVisual Desktop Access (screen reader)
OER	Operational and Environmental Requirement
OCR	Optical Character Recognition
PII	Personally Identifiable Information
PR	Performance Requirement
R	Requirement
RL	Release Requirement
RFT	Reliability and Fault Tolerance Requirement
SCR	Safety-Critical Requirement
SRS	Software Requirements Specification
SR	Security Requirement
SSO	Single Sign-On
TLS	Transport Layer Security
UI	User Interface
UHR	Usability and Humanity Requirement
VnV	Verification and Validation
WCAG	Web Content Accessibility Guidelines

Contents

1 Revision History **i**

2 Symbols, Abbreviations, and Acronyms **iii**

3 Functional Requirements Evaluation **1**

4 Nonfunctional Requirements Evaluation **3**
4.1 Internal Testing Results 4
4.2 System Tests 4

5 Comparison to Existing Implementation **5**

6 Unit Testing **6**
6.1 ImageValidation Module 6
6.2 BackendController Module 7
6.3 AuthenticationService Module 8
6.4 Module 4 10

7 Manual Testing **11**
7.1 UserInterfaceInteractions Module 11
7.2 UserInterfaceAccessibility Module 14
7.3 DataPreProcess Module 17
7.4 ModelOutput Module 19
7.5 AIModelTraining Module 20
7.6 CaptionGeneration Module 20
7.7 InternalMetricCompliance Module 20

8 Manual Testing **20**
8.1 ModelOutput Module 21
8.2 AIModelTraining Module 21
8.3 CaptionGeneration Module 22
8.4 InternalMetricCompliance Module 22

9 Changes Due to Testing **23**
9.1 Narrowing the Scope 23
9.2 Shortening Alt-Text Generation Time 23
9.3 Clearer User Navigation 23

9.4 Future Work 24

10 Automated Testing **24**
10.1 Accessibility Checker - Axe 24
10.2 CI/CD 25

11 Trace to Requirements **25**

12 Trace to Modules **25**

13 Code Coverage Metrics **25**

List of Tables

1 Abbreviations and Acronyms Used in the Verification and Validation Plan iii
2 System Tests Functional Requirements Evaluation 3
3 Internal Evaluation Scores per Evaluator 4
4 System Tests Non-Functional Requirements Evaluation 5
5 ImageValidation Unit Testing Evaluation 7
6 BackendController Module Unit Testing Evaluation 8
7 AuthenticationService Module Unit Testing Evaluation 10
8 Module 4 Unit Testing Evaluation 10
9 UserInterfaceInterfactions Manual Testing Evaluation 14
10 UserInterfaceAccessibility Manual Testing Evaluation 17
11 DataPreprocess Manual Testing Evaluation 19
12 Manual Testing Summary for AI Modules 21
13 Traceability Between Tests and Modules 25

List of Figures

1 Automated Report for "Upload Page" in R4A System 24

This document ...

3 Functional Requirements Evaluation

The following section outlines the evaluation of the system tests created for the functional requirements.

ID	Action	Expected Result	Actual Result	Result
FR-ST1	User uploads image files in different formats to the system (i.e: png, jpeg, pdf, gif)	The system accepts .jpeg and .png files for processing. Files with unsupported formats (.gif, .pdf) are rejected and an error message is displayed.	The system successfully accepted and processed the .jpeg and .png files. The .gif and .pdf files were rejected, and an error message was displayed.	Pass
FR-ST2	User uploads test diagrams (diagram1.png, diagram2.jpeg) to the system and generate alternative text descriptions. Evaluate the generated alt text using the evaluation metrics defined in Appendix 6.1.	The system generates alternative text descriptions for each uploaded image. The descriptions meet the evaluation metrics in Appendix 6.1.	Alt text generated but did not meet the required evaluation metric thresholds. Specifically, the Learning Impact metric scored < 2.	Fail

Continued on next page

(Continued from previous page)

ID	Action	Expected Result	Actual Result	Result
FR-ST3	Generate alternative text for an uploaded image, then use screen readers (NVDA, JAWS, and VoiceOver) to read the generated alternative text aloud.	The generated alternative text is fully read aloud by common screen readers (NVDA, JAWS, VoiceOver) without truncation, misreading, or formatting errors.	The screen reader VoiceOver successfully read the generated alternative text fully and clearly.	Pass
FR-ST4	Edit the generated alternative text by adding words, deleting sentences, and modifying phrasing, then save the changes.	The system accurately reflects all user edits and stores the updated alternative text without data loss or formatting errors.	The user's edits (added, deleted, and modified text) were correctly reflected and saved. The updated alternative text was stored without data loss or formatting issues.	Pass
FR-ST5	Upload more than `SESSION_HISTORY_MAX` images sequentially in the same session, then navigate or refresh the session interface to view the upload history.	Only the `SESSION_HISTORY_MAX` most recent uploads and alt text are visible.	Recent uploads up to the limit were displayed correctly.	Pass

Continued on next page

(Continued from previous page)

ID	Action	Expected Result	Actual Result	Result
FR-ST6	Attempt to log in using both valid McMaster credentials (McMaster email and password) and invalid credentials (non-McMaster email or incorrect password).	Users with valid McMaster University credentials are granted access to the system. Login attempts with invalid credentials are rejected and an error message is displayed.	Login with valid McMaster credentials was successful and access to the system was granted. Login attempts using invalid credentials were rejected and an error message was displayed.	Pass
FR-ST7	Log into the system and remain inactive for longer than `SESSION_TIMEOUT`, then attempt to access the system again.	After `SESSION_TIMEOUT` of inactivity, the session expires automatically. The user is redirected or prompted to log in again before accessing the Reading4All system.	Session did not expire and system remained accessible after `SESSION_TIMEOUT` of inactivity.	Fail

Table 2: System Tests Functional Requirements Evaluation

4 Nonfunctional Requirements Evaluation

The following section reports on the nonfunctional requirements Team 22 evaluated and its results.

4.1 Internal Testing Results

Five evaluators assessed the generated alt text using the metrics defined in Table 3. Each evaluator rated the output on Sufficiency of Description, Length Appropriateness, Accessibility/Usability, and Learning Impact. Table 3 summarizes the scores collected.

Table 3: Internal Evaluation Scores per Evaluator

Evaluator	**Sufficiency (1–3)**	**Length (1–3)**	**Accessibility (0–3)**	**Learning (0–3)**	**Notes**
Person 1	2	3	3	1	N/A
Person 2	2	3	3	2	N/A
Person 3	2	3	3	1	N/A
Person 4	2	2	3	1	N/A
Person 5	2	2	3	1	N/A
Mean	**2.0**	**2.6**	**3.0**	**1.2**	—
Acceptable	≥ 3	≥ 3	≥ 2	≥ 2	—

The results indicate that Accessibility/Usability consistently met or exceeded the acceptable threshold across all evaluators, with a mean score of 3.0. Length Appropriateness averaged 2.6, falling slightly below the acceptable threshold of 3. Sufficiency of Description scored uniformly at 2.0, below the acceptable threshold of 3, suggesting that the generated alt text does not yet convey sufficient information to fully achieve the intended objective. Learning Impact averaged 1.2, also below the acceptable threshold of 2, indicating that improvements are needed to better support user understanding in learning contexts.

4.2 System Tests

The following section outlines the evaluation of the system tests created for the nonfunctional requirements.

ID	Action	Expected Result	Actual Result	Result
NFR-ST1	Run the system website through Axe (third-party service) to check for WCAG 2.1 AA compliance.	There are no issues for all website pages	Some JSON results reported 'moderate' or 'best practices' issues in the summary.	Fail
NFR-ST2	System website is hosted on Vercel (third-party service).	Website is in service and usable with minimal to no crashes.	Website is fully functional on the Vercel server. No crashing detected.	Pass
NFR-ST3	System website is hosted on a testing environment.	Core functions (uploading an image, text generation, and user navigation) function as intended.	Core functionality remains intact on testing environment.	Pass
NFR-ST4	System website is deployed into staging environment.	System is fully ready for DEMO deliverable as per Capstone's March timeline.	Some tests failed, indicating system is not ready for staging environment.	Fail
NFR-ST5	When executing actions in the system, logs are available for API interactions and standard workflows	Logs are visible for any crash or API call.	Logging module was implemented and appropriate level of logs are shown.	Pass

Table 4: System Tests Non-Functional Requirements Evaluation

5 Comparison to Existing Implementation

This section will not be appropriate for every project.

6 Unit Testing

The following are the evaluation of the unit tests per module.

6.1 ImageValidation Module

ID	Action	Expected Result	Actual Result	Result
IMGVAL-UT1	Validate request with missing image.	The module returns `"MISSING_IMAGE"` to indicate the missing image.	The module returned `"MISSING_IMAGE"`.	Pass
IMGVAL-UT2	Validate request with invalid file type.	The module returns `"INVALID_FILE_TYPE"` to indicate that the uploaded file is not a valid image type.	The module returned `"INVALID_FILE_TYPE`	Pass
IMGVAL-UT3	Validate request with an image that exceeds the size limit.	The module returns `"FILE_SIZE_INVALID"` to indicate that the uploaded image exceeds the size limit.	The module returned `"FILE_SIZE_INVALID`	Pass
IMGVAL-UT4	Validate request with a corrupted image file.	The module returns `"UNAUTHORIZED_ACCESS_OR_CORRUPTED"` to indicate that the uploaded image cannot be accessed.	The module returned `"UNAUTHORIZED_ACCESS_OR_CORRUPTED"`.	Pass

Continued on next page

(Continued from previous page)

ID	Action	Expected Result	Actual Result	Result
IMGVAL-UT5	Validate request with a valid image file.	The module returns `"Success"` to indicate that the uploaded image meets system requirements and has been successfully uploaded.	The module returned `"Success"`.	Pass

Table 5: ImageValidation Unit Testing Evaluation

6.2 BackendController Module

ID	Action	Expected Result	Actual Result	Result
BCONT-UT1	Send image validation request with valid image data through the backend controller.	The backend controller forwards the image to the Image Validation module and returns the validation results.	The backend controller correctly returns `Success` after calling the image validation module.	Pass
BCONT-UT2	Send alt text generation request with image data and a valid session ID through the backend controller.	The backend controller forwards the request to the Alt Text Generation module and returns the generated alt text.	The backend controller correctly returns the generated alt text of `"Generated alt text"`.	Pass

Continued on next page

(Continued from previous page)

ID	Action	Expected Result	Actual Result	Result
BCONT-UT3	Send alt text history retrieval request with a valid session ID through the backend controller.	The backend controller forwards the session ID to the Alt Text History module and returns the retrieved history.	The backend controller correctly returns the retrieved history of `["Alt text 1", "Alt text 2"]`.	Pass
BCONT-UT4	Send alt text editing request through the backend controller.	The backend controller forwards the request to the Edit Alt Text module and returns the result.	The backend controller correctly returns the editing result of `"Success"`.	Pass

Table 6: BackendController Module Unit Testing Evaluation

6.3 AuthenticationService Module

ID	Action	Expected Result	Actual Result	Result
AUTH-UT1	Send a request with a valid email and password to the AuthenticationService.	The authentication service successfully creates a new user account and returns the user ID and access token.	The authentication service correctly returns the user ID and access token, i.e., {`"user_id": "test_user_id", "access_token": "test_access_token"`}	Pass

Continued on next page

(Continued from previous page)

ID	Action	Expected Result	Actual Result	Result
AUTH-UT2	Send request with invalid email format to the AuthenticationService	The authentication service raises a `ValueError` with the message "Email must be a @mcmaster.ca address". The supabase client is not called.	The authentication service correctly raises a `ValueError` with the message "Email must be a @mcmaster.ca address" and the supabase client is not called.	Pass
AUTH-UT3	Send a request with a password shorter than 8 characters to the AuthenticationService.	The authentication service raises a `ValueError` with the message "Password must be at least 8 characters". The Supabase client is not called.	The authentication service correctly raises a `ValueError` with the message "Password must be at least 8 characters" and the Supabase client is not called.	Pass
AUTH-UT4	Send a request with empty email and password to the AuthenticationService.	The authentication service raises a `ValueError` with the message "Email must be a @mcmaster.ca address". The Supabase client is not called.	The authentication service correctly raises a `ValueError` with the message "Email must be a @mcmaster.ca address" and the Supabase client is not called.	Pass

Continued on next page

(Continued from previous page)

ID	Action	Expected Result	Actual Result	Result
AUTH-UT5	Send a request with `STUDENT@mcmaster.ca` as the email to the AuthenticationService.	The authentication service correctly converts the email to lowercase before processing.	The authentication service correctly converts the email to lowercase before processing. The Supabase client is called with `student@mcmaster.ca`	Pass
AUTH-UT6	Send a request with an existing email to the AuthenticationService.	The authentication service raises a `ValueError` with the message "Email is already in use". A verification email is not sent to the user	No error was raised and the user received a Verification email.	Fail

Table 7: AuthenticationService Module Unit Testing Evaluation

6.4 Module 4

ID	Action	Expected Result	Actual Result	Result
M-UT1	action	expected result	evaluate	Pass
M-UT2	action	expected result	evaluate	Pass

Table 8: Module 4 Unit Testing Evaluation

7 Manual Testing

7.1 UserInterfaceInteractions Module

The following are the manual tests for this module.

UII-MT 1. **Initial State:** User is logged in and the main screen is displayed.
Input: User selects and uploads an image file using the upload button.
Test Steps:

(a) Click the Choose File button.

(b) Select a valid image file (.png or .jpeg).

(c) Submit the upload.

Expected Results: The system accepts the image and sends it to the backend for processing. The uploaded image appears in the interface and alt-text generation begins.
Failure Condition: The image fails to upload, the UI freezes, or the system does not start alt-text generation.

UII-MT 2. **Initial State:** User is logged in and the main screen is displayed.
Input: User attempts to upload an unsupported file type or file exceeding the maximum size.
Test Steps:

(a) Click the Choose File button.

(b) Select an unsupported file (e.g., .gif, .pdf) or a file larger than the allowed size.

(c) Submit the upload.

Expected Results: The system rejects the file and displays an error message such as "File type not supported" or "File too large."
Failure Condition: The system accepts unsupported files or fails to display an error message.

UII-MT 3. **Initial State:** Generated alt text is displayed on the screen.
Input: User presses the Copy button to copy the generated alt text.
Test Steps:

(a) Generate alt text for an uploaded image.

(b) Click the Copy Alt Text button.

(c) Paste the clipboard contents into another application (e.g., a text editor).

Expected Results: The copied text matches the generated alt text exactly.
Failure Condition: The clipboard is empty or the copied text is incomplete or incorrect.

UII-MT 4. **Initial State:** Generated alt text is visible and editable on the screen.
Input: User edits the alt text and saves the changes.
Test Steps:

(a) Modify the alt text by adding, deleting, or changing words.

(b) Click the Save Edits button.

Expected Results: The updated alt text is displayed and stored correctly without formatting issues.
Failure Condition: The edits are not saved, the text reverts to its original version, or formatting errors occur.

UII-MT 5. **Initial State:** Login screen is displayed.
Input: User enters valid credentials and attempts to sign in.
Test Steps:

(a) Enter valid login credentials.

(b) Click the Sign In button.

Expected Results: The user is authenticated and redirected to the main system interface.

Failure Condition: Valid credentials are rejected or no response occurs after clicking Sign In.

UII-MT 6. **Initial State:** User is logged into the system.
Input: User presses the Sign Out button.
Test Steps:

(a) Click the Sign Out button.

Expected Results: The user session ends and the system redirects the user to the login screen.
Failure Condition: The user remains logged in or can continue accessing the system without logging in again.

The following table shows the evaluation of the userinterfaceinteractions module's manual tests.

ID	Action	Expected Result	Actual Result	Result
UII-MT1	Upload a valid image file (.png or .jpeg) using the Choose File button	Image is accepted and alt-text generation begins	Image uploaded successfully and alt-text generation initiated	Pass
UII-MT2	Upload an unsupported file type or file exceeding the allowed size	System rejects file and displays an appropriate error message	Unsupported file was rejected and an error message was displayed	Pass
UII-MT3	Copy generated alt text using the Copy Alt Text button	Alt text is copied to clipboard accurately	Alt text copied successfully and matched the generated output when pasted	Pass

Continued on next page

(Continued from previous page)

ID	Action	Expected Result	Actual Result	Result
UII-MT4	Edit generated alt text and save changes	System saves and displays updated alt text without formatting issues	Edits were saved correctly and updated text was displayed	Pass
UII-MT5	Enter valid credentials and click Sign In	User is authenticated and redirected to the main interface	Login successful and user was redirected to the main screen	Pass
UII-MT6	Click the Sign Out button	User session ends and login screen is displayed	User session ended and system returned to the login screen	Pass

Table 9: UserInterfaceInterfactions Manual Testing Evaluation

7.2 UserInterfaceAccessibility Module

The following are the manual tests for this module.

UIA-MT 1. **Initial State:** A system screen such as the login screen, main screen, or history screen is loaded.
Input: The tester inspects the rendered page structure using browser accessibility tools or screen reader landmarks.
Test Steps:

(a) Open a system screen.

(b) Inspect the page structure using accessibility inspector or a screen reader landmark list.

(c) Verify that the main content area is identified as the page's main landmark.

Expected Results: The screen contains a valid main landmark (`<main>` or equivalent) for the primary content area.

Failure Condition: No main landmark is present, or the primary content is not identified correctly.

UIA-MT 2. **Initial State:** The login screen or another screen (main or history screen)has just loaded.
Input: The tester loads the screen and checks where keyboard focus is placed.
Test Steps:

(a) Open the login screen.

(b) Do not click the mouse.

(c) Observe which element receives focus automatically.

(d) Press `Tab` to continue keyboard navigation if needed.

Expected Results: Keyboard focus is moved to the intended first interactive element, such as the username field on the login screen.
Failure Condition: Focus is missing, placed on a non-interactive element, or placed on the wrong control.

UIA-MT 3. **Initial State:** A screen reader is enabled and the user performs an action that changes the UI.
Input: Trigger a UI change such as loading the login screen, generating alt text, or navigating to history.
Test Steps:

(a) Enable a screen reader such as NVDA, JAWS, or VoiceOver.

(b) Perform an action that changes the interface.

(c) Listen for the announcement made by the system.

Expected Results: A clear announcement describing the UI change is read by the screen reader through the live region.
Failure Condition: No announcement is made, the message is incomplete, or the announcement is unclear.

UIA-MT 4. **Initial State:** A screen containing multiple interactive elements is open.
Input: The tester uses only the keyboard to navigate within the interface.
Test Steps:

(a) Open a screen with interactive controls.

(b) Use `tab` to move between elements where applicable.

(c) Use `Enter` or `Space` to activate the selected control.

Expected Results: Keyboard navigation works correctly within the defined scope. Focus moves predictably and activation occurs without changing application state unexpectedly.
Failure Condition: Tab key do not work, focus movement is inconsistent, or activation fails.

The following table shows the evaluation of the userinterfaceaccessibility module's manual tests.

ID	Action	Expected Result	Actual Result	Result
UIA-MT1	Verify that the loaded screen contains a valid main landmark for primary content	Main content area is identified by a valid main landmark	Main landmark was not present and did not identify the primary content area	Fail
UIA-MT2	Load the screen and check automatic keyboard focus placement	Focus is placed on the intended first interactive element	Initial focus was correctly placed on the expected interactive control	Pass
UIA-MT3	Trigger a UI change while a screen reader is enabled	Screen reader announces the interface change clearly	Announcement was read clearly by the screen reader after any UI update	Pass

Continued on next page

(Continued from previous page)

ID	Action	Expected Result	Actual Result	Result
UIA-MT4	Use arrow keys, Home/End, and Enter/Space to navigate and activate controls	Keyboard navigation works correctly within the interface scope	Keyboard navigation and activation worked as expected without unexpected state changes	Pass

Table 10: UserInterfaceAccessibility Manual Testing Evaluation

7.3 DataPreProcess Module

The following are the manual tests for this module.

DP-MT 1. **Initial State:** The preprocessing function is available and ready to process dataset examples.
Input: A dataset example containing valid image paths and modified alt text.
Test Steps:

(a) Provide a dataset example containing valid image paths and alt text.

(b) Run the preprocess function.

(c) Inspect the returned encoded output.

Expected Results: The function successfully loads images and text and returns encoded tensors containing image and text representations.
Failure Condition: The preprocessing function fails to return encoded outputs or raises an unexpected error.

DP-MT 2. **Initial State:** The preprocessing function is ready to process input images.
Input: Image files that may not originally be in RGB format.
Test Steps:

(a) Provide image paths to the preprocess function.

(b) Run preprocessing.

(c) Inspect the processed images.

Expected Results: All images are converted to RGB format before being passed to the processor.
Failure Condition: Images remain in their original format and are not converted to RGB.

DP-MT 3. **Initial State:** The preprocessing function receives alt text label with leading or trailing whitespace.
Input: Alt text label containing extra whitespace characters.
Test Steps:

(a) Provide alt text label with leading or trailing spaces.

(b) Run preprocessing.

(c) Inspect the processed text values.

Expected Results: Leading and trailing whitespace is removed from the alt text.
Failure Condition: Whitespace remains in the processed alt text.

DP-MT 4. **Initial State:** The preprocessing function is ready to process dataset examples.
Input: A dataset example containing an invalid or unreadable image path.
Test Steps:

(a) Provide a dataset example containing an invalid image path.

(b) Run the preprocess function.

(c) Observe the system response.

Expected Results: The function raises an error indicating that the image cannot be opened or read.
Failure Condition: The function silently proceeds or produces

invalid encoded output.
eeds the maximum length.

The following table shows the evaluation of the datapreprocess module's manual tests.

ID	Action	Expected Result	Actual Result	Result
DPP-MT1	Process valid image paths and alt text labels through the preprocess function	Valid images and text are encoded successfully	Images and alt text were processed successfully and encoded tensors were returned	Pass
DPP-MT2	Provide image paths and run preprocessing to verify RGB conversion	Images are converted to RGB format before encoding	Images were successfully converted to RGB format before processing	Pass
DPP-MT3	Provide alt text containing leading or trailing whitespace	Whitespace is removed from alt text	Alt text whitespace was successfully stripped before encoding	Pass
DPP-MT4	Provide an invalid or unreadable image path to preprocessing	Function raises an error indicating the image cannot be opened	Invalid image path correctly triggered an error during preprocessing	Pass

Table 11: DataPreprocess Manual Testing Evaluation

7.4 ModelOutput Module

No automated unit tests were implemented for this module. Its behavior was verified through the manual tests described in Section 7 (MO-MT1), which confirm that generated alt-text is correctly formatted and displayed to the user.

7.5 AIModelTraining Module

Unit tests were not implemented for this module because the training process operates as part of the model training pipeline rather than as isolated functions. Validation was performed through the manual testing procedures described in Section 7 (AI-MT1).

7.6 CaptionGeneration Module

The functionality of this module was evaluated through the manual tests described in Section 7 (CG-MT1), where uploaded images were used to verify that the system generates meaningful alt-text descriptions.

7.7 InternalMetricCompliance Module

No dedicated unit tests were implemented for this module. Instead, its behavior was verified through the manual tests in Section 7 (IMC-MT1), where generated captions were evaluated against the system's internal quality metrics.

8 Manual Testing

The following section outlines the manual tests performed for the AI modules.

Test ID	Module	Description	Actual Result	Result
MO-MT1	ModelOutput Module	Verify generated alt-text is correctly formatted and displayed to the user.	All assertions pass	Pass
AI-MT1	AIModelTraining Module	Verify that the model training process successfully produces a trained model.	All assertions pass	Pass

Continued on next page

(Continued from previous page)

Test ID	Module	Description	Actual Result	Result
CG-MT1	CaptionGeneration Module	Verify that the system generates descriptive alt-text for uploaded images.	All assertions pass	Pass
IMC-MT1	InternalMetricCompliance Module	Verify that generated captions satisfy internal quality evaluation metrics.	All assertions pass	Pass

Table 12: Manual Testing Summary for AI Modules

8.1 ModelOutput Module

The following are the manual tests for this module.

MO-MT 1. **Initial State:** The system has successfully generated alt-text for an uploaded image.

Input: The backend returns the generated caption to be displayed to the user.

Test Steps:

(a) Upload a valid diagram image.

(b) Wait for the alt-text generation process to complete.

(c) Observe the output displayed on the user interface.

Expected Results: The generated alt-text is correctly formatted and displayed to the user through the interface.

Failure Condition: The generated alt-text is missing, incorrectly formatted, or not displayed to the user.

8.2 AIModelTraining Module

The following are the manual tests for this module.

AI-MT 1. **Initial State:** The training dataset and configuration parameters are available.

Input: The model training process is initiated using the dataset.

Test Steps:

(a) Start the training pipeline.
(b) Load the dataset used for training the model.
(c) Monitor the training process for completion.

Expected Results: The model successfully completes training and produces a trained model artifact used for caption generation.

Failure Condition: Training fails to complete, produces errors, or does not generate a usable trained model.

8.3 CaptionGeneration Module

The following are the manual tests for this module.

CG-MT 1. **Initial State:** A trained model exists and a valid image has been uploaded.

Input: The system processes the uploaded image using the trained model.

Test Steps:
(a) Upload a valid diagram image.
(b) Submit the image for caption generation.
(c) Wait for the caption generation process.

Expected Results: The system generates descriptive alt-text that reflects the content of the image.

Failure Condition: The system fails to generate alt-text, produces irrelevant text, or returns an error.

8.4 InternalMetricCompliance Module

The following are the manual tests for this module.

IMC-MT 1. **Initial State:** A caption has been generated by the CaptionGeneration module.

Input: The generated caption is evaluated using internal quality metrics.

Test Steps:
(a) Generate alt-text from an uploaded image.
(b) Allow the system to run internal metric validation checks.
(c) Observe whether the caption passes the internal quality checks.

Expected Results: The caption is evaluated against internal metrics and confirmed to meet clarity and quality standards.

Failure Condition: The caption fails validation checks or the system does not evaluate the generated caption.

9 Changes Due to Testing

During the internal testing phase, several changes were made to the tool based on feedback from the TA, professor, supervisor and from Team 22. A brief description of each change and its implementation is listed below:

9.1 Narrowing the Scope

While the team had the ambition to make a STEM-alt text generator, this proved to be very expensive in terms of time needed for training and constraints on available computers for AI model tuning. To make up for this, Team 22 narrowed the scope to only "Physics" based diagrams, with a plan to further narrow the image types to a select list. This change would help our AI model learn better, contextual text and in turn produce higher quality, specific alt text that is usable for several stakeholders like students and teachers.

9.2 Shortening Alt-Text Generation Time

The AI model initially took five minutes or more to generate text for one image. After talking to our supervisor, Team 22 was able to decide on a faster model that provided mediocre alt-text generation while adding in indicators for user navigations and processes. We were able to cut down from approximately five minutes per image to 1-2 minutes per image, which significantly reduced user frustration.

9.3 Clearer User Navigation

During testing, Team 22 had found that some user navigation processes were not as clear to a screen-reader user or someone with a visual impairment. Examples include no descriptive 'aria' labels or a misplaced page navigation. To fix this, we tried testing both automatically and manually in several ways including testing the solution with just the keyboard and voice assist. This significantly improved our solution, reduced tester frustration and gave users a clear indication of what was going on behind the scenes.

9.4 Future Work

While our testing showed us many faults in our solution, not all of them could be fixed. For future revisions, Team 22 plans to further increase alt-text quality by adding more specificity and context to generation (using AI hypertuning methods). Conducting user testing has given us sufficient data to know what features and system modules to improve for better usability.

10 Automated Testing

10.1 Accessibility Checker - Axe

The accessibility checks were done using a third-party website, Axe, which helped Team 22 understand gaps in user navigation for people with disabilities. An example of a generated report can be seen below:

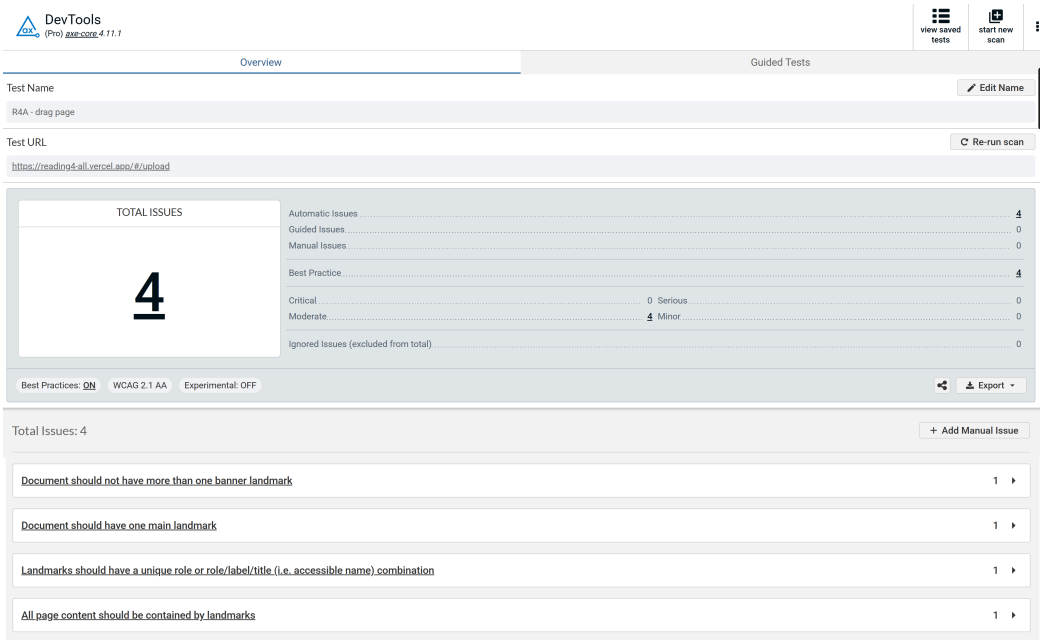

Figure 1: Automated Report for "Upload Page" in R4A System

Reports like these made it easier for the team to identify accessibility oversights and best practices we might have missed and fix them.

10.2 CI/CD

Continuous Integration and Continuous Deployment (CI/CD) practices were used to ensure that changes to the Reading4All system could be integrated and tested efficiently. Whenever updates were pushed to the repository, automated workflows were triggered to verify that the system built successfully and that core components continued to function as expected.

These automated checks helped the team detect issues early in the development process and prevented unstable code from being merged into the main branch. By incorporating CI/CD into the development workflow, Team 22 was able to maintain code quality, streamline collaboration, and ensure that new features or fixes did not unintentionally break existing functionality.

Additionally, the CI/CD pipeline supported reproducibility by ensuring that the same build and testing procedures were executed consistently for each update. This contributed to a more stable and reliable development process for the Reading4All system.

11 Trace to Requirements

12 Trace to Modules

The following table shows the relationship between the tests performed and the modules defined in the Module Guide. Each test verifies the correct functionality of the corresponding module.

Test ID	Module
MO-MT1	ModelOutput Module
AI-MT1	AIModelTraining Module
CG-MT1	CaptionGeneration Module
IMC-MT1	InternalMetricCompliance Module

Table 13: Traceability Between Tests and Modules

13 Code Coverage Metrics

References

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Reflection.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

Team Reflection

1. Which parts of this document stemmed from speaking to your client(s) or a proxy (e.g. your peers)? Which ones were not, and why?

   One aspect that stemmed from speaking with our supervisor is the importance of manual testing over automated testing, specifically the user interface. Our supervisor emphasized the importance of ensuring our interface is accessible and this is highlighted through our various nun-functional tests. We tried our best to put ourselves in the users shoes and understand any difficulties or issues they might face while using the Reading4All application. Furthermore, determining which aspects of the system to unit test verses manually test was determined through discussions with our TA supervisor. This was an aspect we were initialling unsure about; however, after discussions we determined that certain aspects such as the system as a whole, the accessibility, and alt text generation might be better manually tested. The Reading4All system implementation was mainly derived from our previous learnings through courses and industry standards, rather than from our clients and peers.

2. In what ways was the Verification and Validation (VnV) Plan different from the activities that were actually conducted for VnV? If there were

differences, what changes required the modification in the plan? Why did these changes occur? Would you be able to anticipate these changes in future projects? If there weren't any differences, how was your team able to clearly predict a feasible amount of effort and the right tasks needed to build the evidence that demonstrates the required quality? (It is expected that most teams will have had to deviate from their original VnV Plan.)

One way that the VnV plan was different from the activities conducted was that we initially planned to test all our modules, prior to us determining and forming our modules. This caused an issue when trying to actually conduct the VnV, as the modules were designed without having the testing in mind. This led us to have modules that were difficult to directly test and have concrete results for. As a result, one modification to the plan and overall system, is that certain modules had to be refactored as they could not be effectively tested and did not have any code we can point to. Furthermore, our VnV plan was dependent on a specific solution, that was vastly modified throughout the year. The VnV plan should have been written in a more generalized manner such that it would be flexible for any solution. For example, earlier in the year our team thought that a feedback module would be an appropriate way to train our AI model, however, through more research and various experiments, we determined that hypertuning would lead to better results. We believe we could not have anticipated this, as based on our AI knowledge at the time, we believed this would lead to the best design and implementation. Looking back, it's important for us to make a testing plan that depends more on the results and functionality that our system will exhibit rather than how it will be implemented as this often changes.

Fiza's Reflection

1. What went well while writing this deliverable?

   One aspect that went well while writing this deliverable was the clear structure provided by the VnV template and the previous planning documents. Since the system architecture, modules, and test plans were already defined in earlier deliverables such as the Module Guide and VnV Plan, it was easier to connect the implemented tests to the system components. Collaboration within the team also worked well, as

each member was responsible for specific modules and testing sections, which allowed the work to be divided efficiently and integrated into a consistent report format.

2. What pain points did you experience during this deliverable, and how did you resolve them?

   One challenge during this deliverable was ensuring consistency between the different sections of the report, such as aligning manual tests, system tests, and module traceability with the modules defined in the Module Guide. Another difficulty was formatting complex LaTeX tables and ensuring they compiled correctly while maintaining a consistent style across the document. These issues were resolved through iterative debugging of the LaTeX code and by coordinating with teammates to ensure the testing sections aligned with the previously defined modules and test cases.

Moly Mikhail's Reflection

1. What went well while writing this deliverable?

   I think many aspects went well while writing this deliverable. Writing the unit tests for the backend was a smooth process as I was highly familiar with the code base and the different branches to be covered. Another aspect that went well was documenting our unit test results, this also provided another opportunity for me to review the unit tests so far and to determine if any tests were missing and add them.

2. What pain points did you experience during this deliverable, and how did you resolve them?

   One pain point I experienced writing this deliverable is determining the difference between the unit test section and automated testing. At first I was confused on the difference; however, after discussing with our TA, it was clearly they are highly related. Furthermore, it was also difficult determining which aspects of the system should be uni tested, verses manual tested. This was also cleared up through discussions with Chris. We decided the for integration testing it would be more effective to do it manually rather than to implement Selenium.